



Cairo University

Faculty of Engineering

CMPS303 – Operating systems

Phase 2 – Report

Submitted By:

Abdelrahman Ali	1220062
Abdelrahman Shawky	1220126
Abdelrahman Zakria	1220067
Raghad Waleed	1220123

MMU (Memory management unit):

Virtual Memory per process

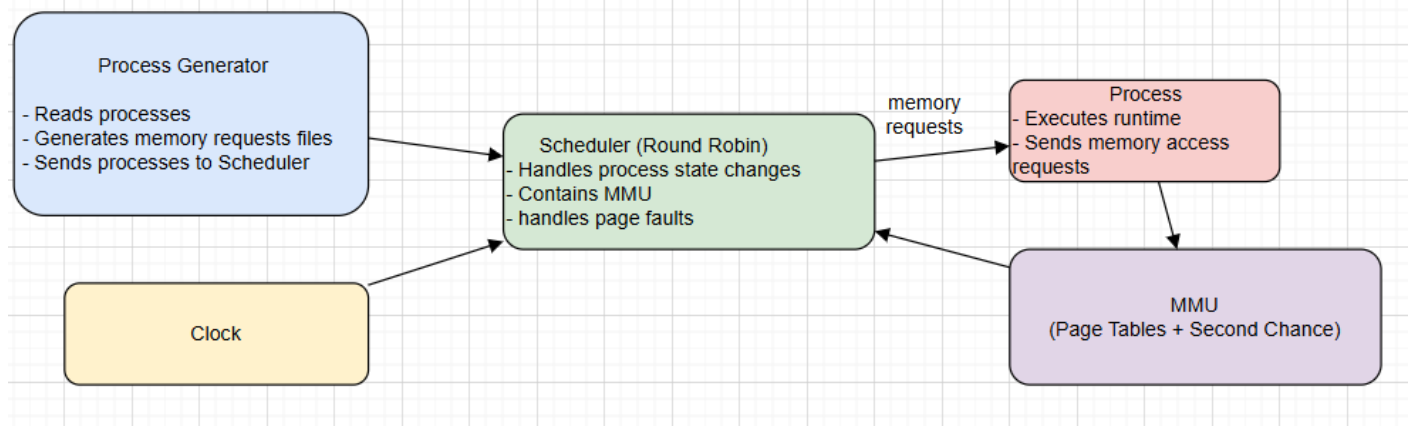
- Fixed page size = **16 bytes**
- Physical memory = **32 frames**
- Page table per process

Each process has its own **page table** stored in its PCB.

Table-summary:

Item	Value
Address space	10-bit
Max memory	1024 bytes
RAM size	512 bytes
Page size	16 bytes
Number of physical pages	$512 / 16 = 32$ pages
Disk access time	10 cycles
Memory access	1 cycle
Swapping algorithm	Second Chance
Scheduling algorithm	Round Robin ONLY

Block Diagram



The system starts with the Process Generator, which reads the input processes file, creates memory request files for each process, and sends processes to the Scheduler at their arrival times. The Clock provides a global notion of time that synchronizes all components. The Scheduler (Round Robin) controls CPU execution, switches processes based on the time quantum, and manages process states. While a process is running, it sends memory access requests to the Scheduler, which forwards them to the MMU. The MMU, using page tables and the Second Chance replacement algorithm, translates virtual addresses to physical addresses, handles page faults, and updates memory state. The Scheduler then replies to the process with the translated address, allowing execution to continue. This loop repeats until all processes finish execution.

Memory addressing:

Each process uses virtual addresses, not physical memory directly. A virtual address is divided into two parts:

- VPN (Virtual Page Number): Identifies which virtual page is being accessed.
- Offset: Identifies the exact location inside that page.

The MMU uses the VPN to look up the corresponding frame number in the process's page table. If the page is not in memory, a page fault occurs and the MMU loads the page into a free frame or replaces another page using the Second Chance algorithm. The physical address is then formed by combining the frame number with the offset.

Page replacement – Second Chance Algorithm:

- Improves FIFO by avoiding immediate eviction
- Approximates Least Recently Used (LRU)
- Efficient and simple

How it works

1. If a page is not in memory → **Page Fault**
2. Search for a free frame
3. If none exists:
 - Check reference bit
 - Give page a “second chance”
 - Evict if reference bit = 0
4. Update page table and RAM metadata

Memory Requests Handling

Each process:

- Reads from request_<pid>.txt
- Sends requests in the form: <time> <virtual_address> <r/w>
- Waits for scheduler response

The scheduler:

- Translates address
- Detects page faults
- Handles swapping
- Responds with physical address

Page Fault Handling

When a page fault occurs:

1. Process is stopped
2. Page replacement is triggered
3. Disk delay is simulated
4. Page is loaded into memory
5. Process resumes execution

All events are logged in memory.log.

Output Files

scheduler.log

Contains:

- Process state transitions
- Arrival time
- Remaining time
- Waiting time
- Turnaround time (TA)
- Weighted turnaround time (WTA)

scheduler.perf

Contains:

- CPU utilization
- Average waiting time
- Average WTA
- Standard deviation of WTA

memory.log

Contains:

- Page faults
- Page allocations
- Page swaps
- Page load times

Link to repo: <https://github.com/AbdelrahmanMShawki/OS-project>